Deep Learning for Visual Computing - Assignment 1

Andjelija Vasiljevic - 12450320
Alicia Schartmann - 12331461

May 2026

What is the purpose of a loss function? What does the cross-entropy measure? Which criteria must the ground-truth labels and predicted class-scores fulfill to support the cross-entropy loss, and how is this ensured?

A loss function defines how well a models predictions match the true targets. Its a numerical measure of error that the training process tries to minimize. Without it, there is no objective signal to guide learning.
Cross-entropy measures the difference between the true distribution of labels and the predicted probability distribution produced by the model. If the model assigns high probability to the correct class, the loss is low, if it assigns low probability, the loss becomes large. To correctly use cross-entropy, the ground-truth labels must represent a valid distribution and predicted class scores must be probabilities (nonnegative, sum to 1). This is ensured by encoding labels properly and applying a softmax function to model outputs. In practice, PyTorchs `CrossEntropyLoss` expects raw logits rather than probabilities for numerical stability and integer class indices instead of one-hot vectors.

What are the goals of data augmentation, regularization, and early stopping? How exactly did you use these techniques (hyperparameters, combinations) and what were your results (train, val and test performance)? List all experiments and results, even if they did not work well, and discuss them.

Data augmentation intentionally expands the training set's effective variety by randomly transforming inputs, forcing the model to learn generalizable features rather than memorizing pixel patterns. *Regularization* (here: weight decay) penalizes large parameter magnitudes via an ℓ_2 penalty, discouraging the model from fitting noise. *Early stopping* halts training once no validation improvement is observed for a given number of consecutive checks (*patience*), preventing over-optimization beyond the point of best generalization.

All ResNet18 experiments used AdamW (lr = 10^{-3}, `amsgrad=True`), ExponentialLR ($\gamma = 0.9$), batch size 128, and 30 epochs with validation every 5 epochs. For augmentation, `RandomCrop(32, padding=4)` and `RandomHorizontalFlip(p=0.5)` were applied to the training set only; validation and test sets received only type conversion and ImageNet normalization. Weight decay was set to 5×10^{-4} when regularization was enabled, and 0 otherwise. Four combinations were tested: no augmentation/regularization, regularization only, augmentation only, and both combined. Early stopping used a patience of two validation checks (= 10 epochs), but was never triggered as validation mPCAcc improved monotonically across all runs; it was therefore set to *False* as default for all subsequent experiments.

The results are summarized in Table 1. Without augmentation, the model significantly overfits: the best validation mPCAcc stagnates at about 76.5% while the training mPCAcc approaches 100% (loss $\approx$ 0.0002), resulting in a test mAcc of only $\approx$ 75.9% (no regularization) and $\approx$ 75.6% (with regularization). Surprisingly, adding weight decay by itself—without augmentation—offers no discernible benefit and even marginally lowers test accuracy, indicating that ℓ_2 regularization is not enough to close the significant train-val gap brought on by the discrepancy between the training and evaluation distributions.

The impact of augmentation is far greater. The training loss increases to 0.332 and train mPCAcc decreases to 87.1% with augmentation and no regularization, indicating that the training task has gotten more difficult and the model is no longer able to memorize training images. Accordingly, the optimal validation mPCAcc jumps to 82.2% and test mAcc to 81.9%—more than six percentage points better than the baseline without augmentation. Weight decay adds nothing once augmentation functions as a powerful implicit regularizer, as evidenced by the nearly same training dynamics and slightly lower test performance (81.4%) that result from adding regularization on top.

Table 1: Comparison of data augmentation and regularization combinations on ResNet-18 trained for 30 epochs on CIFAR-10 with AdamW optimizer and ExponentialLR scheduler.

		Train (final epoch)			**Validation (best epoch)**			**Test**		
Augmentation	**Regularization**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**
✓	✗	0.3318	0.8818	0.8818	0.5643	**0.8219**	**0.8223**	**0.5690**	**0.8187**	**0.8187**
✓	✓	0.3255	0.8837	0.8837	**0.5558**	0.8206	0.8209	0.5726	0.8136	0.8136
✗	✗	0.0002	1.0000	1.0000	1.5757	0.7649	0.7649	1.6335	0.7587	0.7587
✗	✓	0.0002	1.0000	1.0000	1.6352	0.7616	0.7616	1.6309	0.7563	0.7563

What is the purpose of the training, validation, and test sets and why do we need all of them?

These findings demonstrate important relevance of the validation set: in the no-augmentation condition, a near-perfect training score corresponds with the worst generalization, making the train accuracy alone extremely deceptive. Since the test set must remain hidden until the very end, it was crucial to track val mPCAcc during training and save the checkpoint with the best validation performance in order to choose the optimal model.

(Please refer to the appendix for more experiments for ResNet18 like optimizer and learning rate scheduler.)

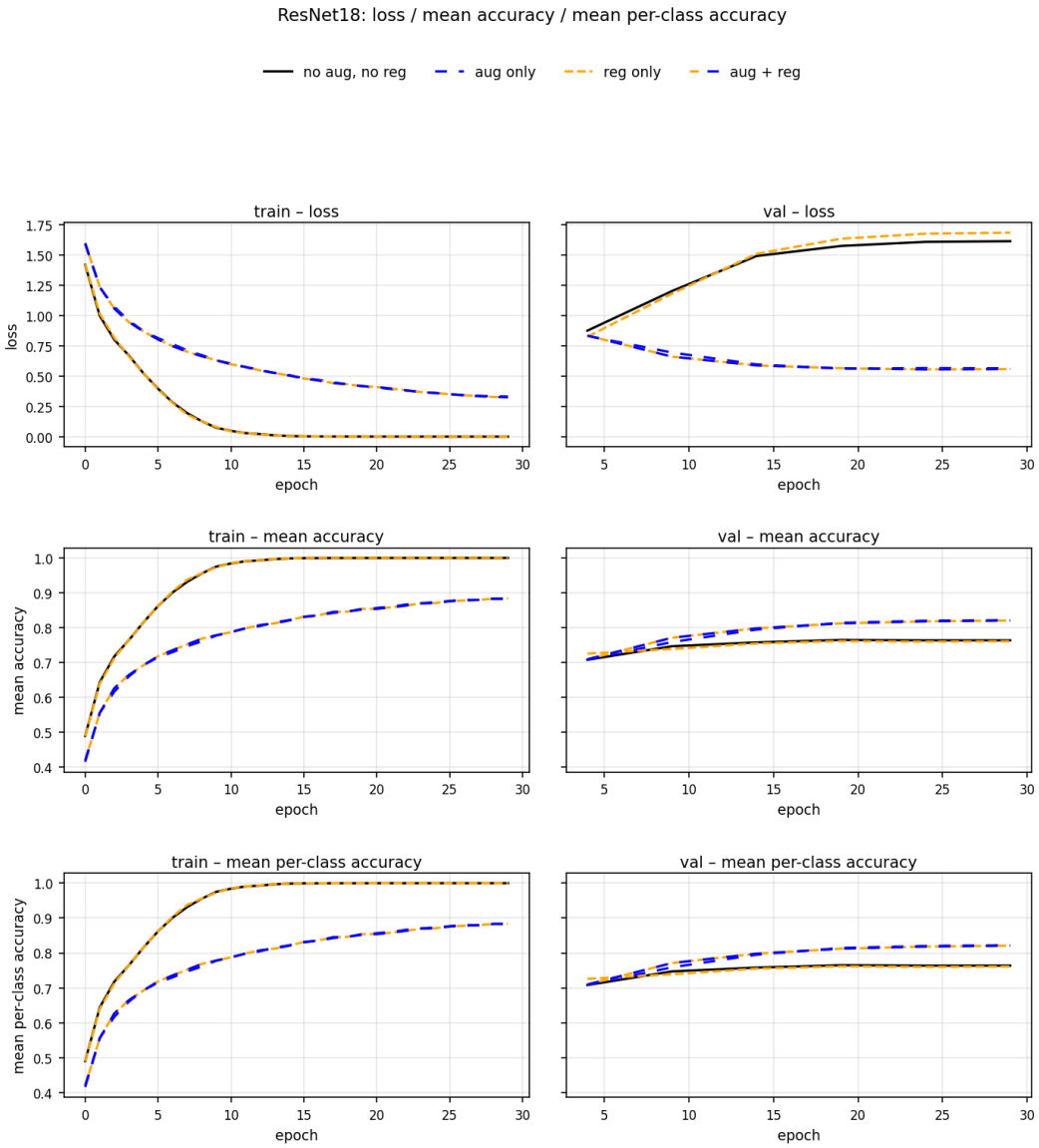


Figure 1: Training and validation metrics for ResNet18 across aug/reg

What are the key differences between ViTs and CNNs? What are the underlying concepts, respectively? Give the source of your ViT model implementation and point to the specific lines of codes (in your code) where the key concept(s) of the ViT are and explain them.

CNNs and ViTs process images in fundamentally different ways: CNNs slide small learned filters across local neighborhoods, while ViTs split the image into patches and let them interact globally through attention. In our `cnn.py`, four `nn.Conv2d` blocks (ll. 18–34) with

shared-weight kernels and `MaxPool2d` implement this directly, building translation equivariance and a hierarchical receptive field into the architecture. A ViT has no such built-in locality, which is why training one from scratch typically requires more data or stronger augmentation than a CNN of comparable size.

Our ViT (`vit.py`, l.34) imports `SimpleViT` from lucidrains/vit-pytorch (MIT; Beyer et al. 2022 [1]); ll.59–68 configure it for CIFAR-10. The `simple_vit.py` contains the four defining ViT operations: *patch embedding* (`to_patch_embedding` Sequential at ll. 90–95, applied at l. 113: `Rearrange + LayerNorm + Linear + LayerNorm` maps each 4×4 patch to a 256-dim token), *positional encoding* (`posemb_sincos_2d` defined ll. 12–21, instantiated ll. 97–101, added to every token at l. 114; fixed 2-D sin–cos), *multi-head self-attention* (the `Attention` class, ll. 37–62: scaled $\texttt{q @ k}^\top$ at l. 56, softmax at l. 58, `@ v` at l. 60; reshape to 8 heads at l. 54), and a *classification head* (`linear_head` defined at l. 108; `x.mean(dim=1)` at l. 117 and `Linear` $\rightarrow$ 10 logits at l. 120; no `[CLS]` token in SimpleViT). We use CosineAnnealingLR as default for the ViT training because exponential decay reduces the learning rate too aggressively in the early epochs, when a from-scratch ViT, which lacks the convolutional inductive biases of a CNN, still needs sustained updates to learn meaningful attention patterns.

Table 2: Comparison of ResNet18, custom CNN, and ViT without data augmentation and without regularization, all trained for 30 epochs on CIFAR-10 with AdamW optimizer and ResNet18 and CNN with ExponentialLR scheduler and ViT with CosineAnnealing scheduler.

	Train (final epoch)			**Validation (best epoch)**			**Test**		
Model	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**
ResNet18	0.0002	1.0000	1.0000	1.5757	0.7649	0.7649	1.6335	0.7587	0.7587
CNN	0.0051	0.9997	0.9997	**0.9552**	**0.8088**	**0.8089**	**0.9719**	**0.8024**	**0.8024**
ViT	0.7181	0.7478	0.7478	1.0875	0.6287	0.6285	1.0867	0.6203	0.6203

A Additional Results: Optimizer and Learning rate scheduler

Table 3 compares Adam, AdamW, and SGD on the baseline setting (no augmentation, no regularization). AdamW achieves the best overall results with the lowest validation loss (1.576), highest validation mPCAcc (76.49%), and best test mAcc (75.87%), while Adam performs comparably on the test set (75.71%); SGD lags behind both adaptive optimizers by roughly six percentage points on the test set (70.97%), likely because momentum-based gradient descent requires more epochs or careful learning rate tuning to match the convergence speed of adaptive methods.

Table 3: Comparison of optimizers (Adam, AdamW, SGD) on ResNet-18 trained for 30 epochs on CIFAR-10.

	Train (final epoch)			**Validation (best epoch)**			**Test**		
Optimizer	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**
Adam	0.0006	0.9999	0.9999	1.9325	0.7588	0.7589	1.9291	0.7571	0.7571
AdamW	0.0002	1.0000	1.0000	**1.5757**	**0.7649**	**0.7649**	**1.6335**	**0.7587**	**0.7587**
SGD	0.0005	1.0000	1.0000	1.8973	0.7084	0.7085	1.8885	0.7097	0.7097

Table 4 compares CosineAnnealingLR, ExponentialLR, and StepLR, all using AdamW on the same baseline setting. CosineAnnealingLR yields the highest validation and test mAcc (77.73% and 76.89% respectively), benefiting from its smooth gradual decay that keeps the learning rate larger for longer; StepLR achieves the lowest validation and test loss (1.119 and 1.143) due to its aggressive drops at epochs 10 and 20, but does not translate this into the best accuracy, while ExponentialLR falls between the two in all metrics.

Table 4: Comparison of learning rate schedulers (CosineAnnealingLR, ExponentialLR, StepLR) on ResNet-18.

	Train (final epoch)			**Validation (best epoch)**			**Test**		
Scheduler	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**
CosineAnnealingLR	0.0003	1.0000	1.0000	1.5154	**0.7773**	**0.7772**	1.5763	**0.7689**	**0.7689**
ExponentialLR	0.0002	1.0000	1.0000	1.5757	0.7649	0.7649	1.6335	0.7587	0.7587
StepLR	0.0029	0.9999	0.9999	**1.1188**	0.7765	0.7765	**1.1434**	0.7672	0.7672

B Additional Findings: ViT and CNN

Table 5: Comparison of data augmentation and regularization combinations on ViT trained for 30 epochs on CIFAR-10 with AdamW optimizer and CosineAnnealing scheduler.

		Train (final epoch)			**Validation (best epoch)**			**Test**		
Augmentation	**Regularization**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**
✓	✗	1.1304	0.5923	0.5924	1.1271	0.5947	0.5943	1.1300	0.5943	0.5943
✓	✓	1.0021	0.6410	0.6410	**1.0162**	**0.6385**	**0.6383**	**1.0190**	**0.6295**	**0.6295**
✗	✗	0.7181	0.7478	0.7478	1.0875	0.6287	0.6285	1.0867	0.6203	0.6203
✗	✓	0.6918	0.7592	0.7592	1.1373	0.6099	0.6098	1.1435	0.6069	0.6069

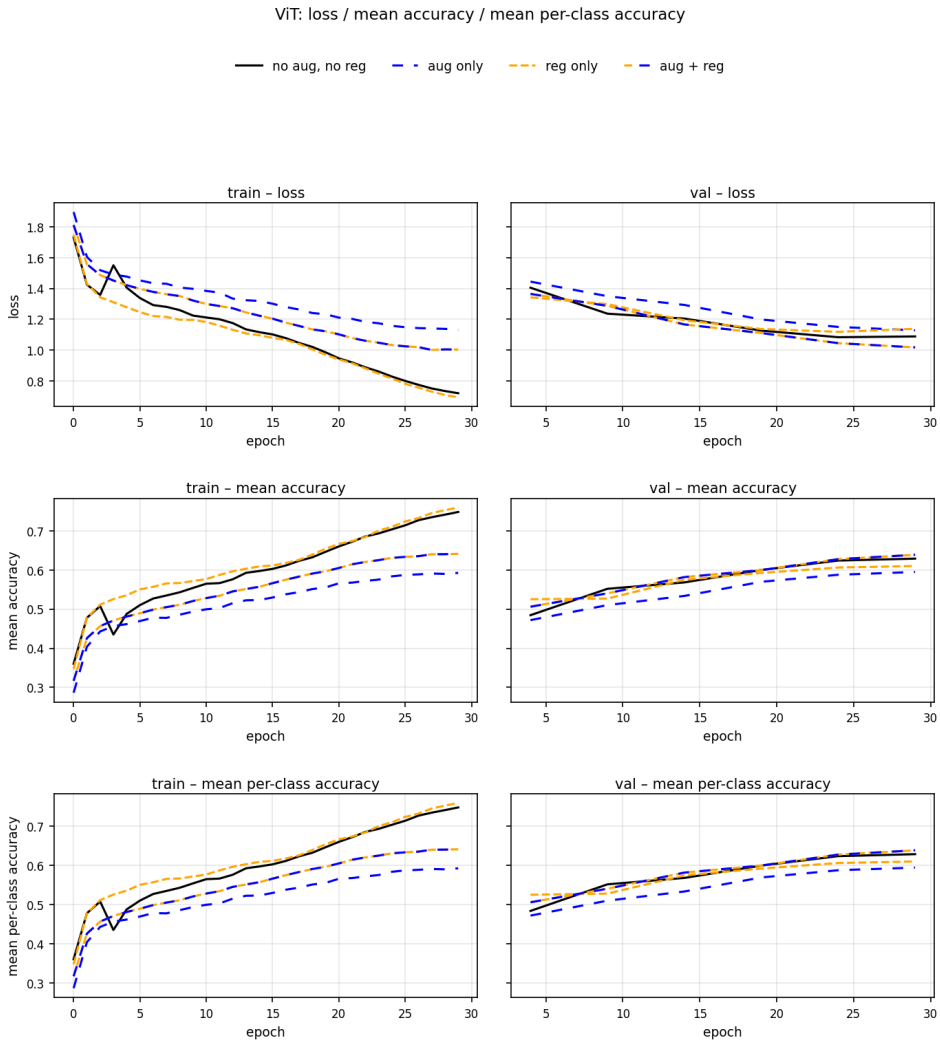


Figure 2: Training and validation metrics for ViT across aug/reg

Table 6: Comparison of data augmentation and regularization combinations on the custom CNN trained for 30 epochs on CIFAR-10 with AdamW optimizer and ExponentialLR scheduler.

		Train (final epoch)			Validation (best epoch)			Test		
Augmentation	**Regularization**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**
✓	✗	0.3769	0.8713	0.8713	**0.4572**	**0.8472**	**0.8474**	**0.4626**	**0.8424**	**0.8424**
✓	✓	0.3863	0.8645	0.8645	0.4583	0.8439	0.8441	0.4730	0.8399	0.8399
✗	✗	0.0051	0.9997	0.9997	0.9552	0.8088	0.8089	0.9719	0.8024	0.8024
✗	✓	0.0047	0.9996	0.9996	0.9431	0.8114	0.8114	0.9744	0.8044	0.8044

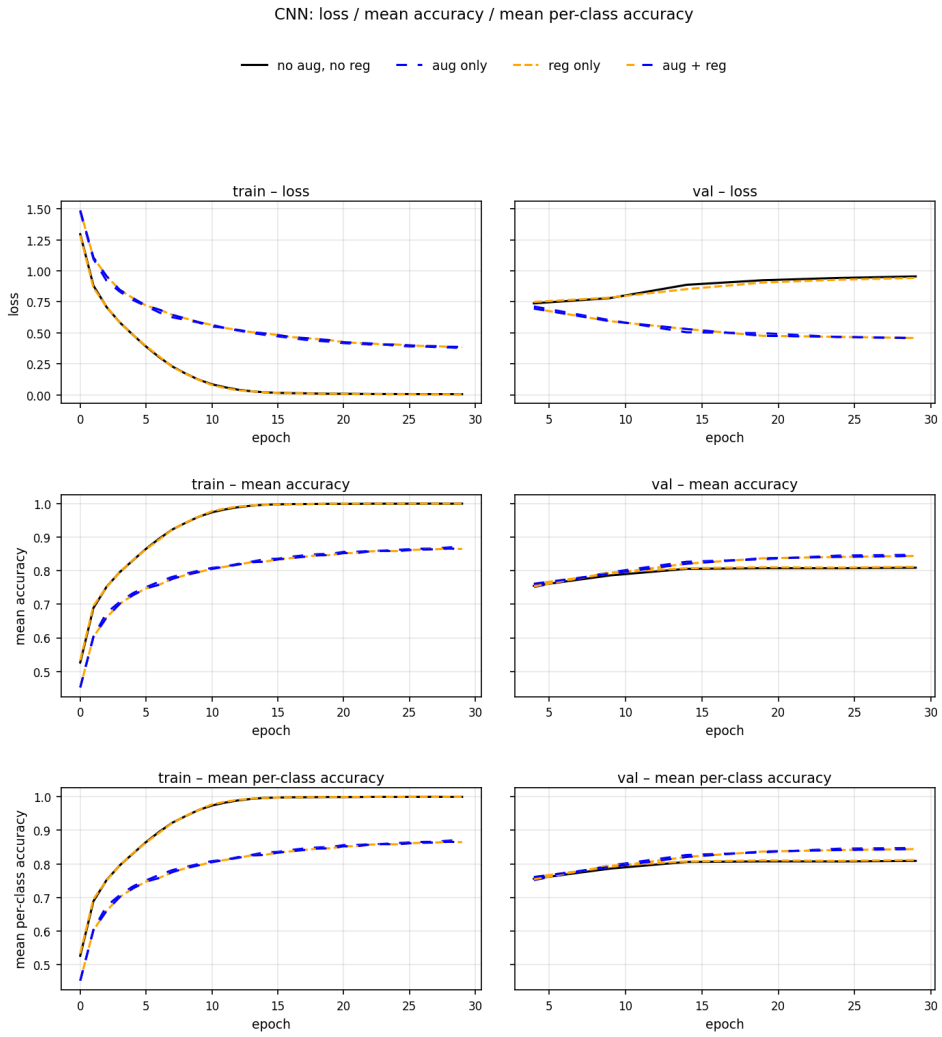


Figure 3: Training and validation metrics for CNN across aug/reg

C Extended Findings

Table 7: Comparison of training with and without early stopping on ResNet-18 trained for 30 epochs on CIFAR-10.

	Train (final epoch)			Validation (best epoch)			Test		
Setup	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**	**Loss**	**mAcc**	**mPCAcc**
Without Early Stopping	0.0002	1.0000	1.0000	**1.5757**	0.7649	0.7649	**1.6335**	**0.7587**	**0.7587**
With Early Stopping	0.0002	1.0000	1.0000	1.5937	**0.7660**	**0.7657**	1.7054	0.7542	0.7542

References

[1] Lucas Beyer, Xiaohua Zhai, and Alexander Kolesnikov. Better plain vit baselines for imagenet-1k. *arXiv preprint arXiv:2205.01580*, 2022.